Uncertainty-Aware Verification of Number-Based Chart Claims in Student Technical Reports

Harsh Raj (2025202005) Hritik Ranjan (2025202031)
Karan Thayat (2025202008) Vikash Kumar (2025201012)

LMA Group Project 2026, Project Proposal October 2026

Abstract

While grading lab reports, a TA has to check whether the numbers written in the text agree with the charts in the same report, which takes a lot of time. Vision-language models can read charts, but only approximately, and they give a verdict even when the reading is not precise enough to judge. In grading this is risky, since a misread chart can become a wrong accusation on a student. We propose an assistant that reads each value using several tools, estimates how accurate its own readings are, and flags a sentence only if the mismatch is clearly bigger than this reading error. If it cannot tell, the case is passed to the TA. We will test whether this gives fewer wrong flags than systems that always answer.

1 Project title

Uncertainty-Aware Verification of Number-Based Chart Claims in Student Technical Reports

2 Domain and target user

Our domain is education, more specifically the grading of lab and project reports which contain charts. The main users are TAs and instructors, who have to check that the numbers in the text match the figures while grading. Students can also use it to check their draft before submission.

3 Problem statement

In technical courses, students submit reports in which the numbers in the text should match their own charts. In practice they often don't, e.g. a number is copied from an older run, an axis is misread, or a value is rounded too much. For a TA with dozens of reports, checking each sentence against each chart by hand is too slow. Vision-language models can help here, but they read chart values only approximately and they tend to say that a claim is supported. The existing checkers also give a verdict every time, so if the model misreads a chart we get either a missed mistake or a wrong accusation. Of the two, the wrong accusation is worse in grading, because the student can lose marks for a sentence that was correct. So we want to build a checker which knows how accurate its own chart reading is, flags a claim only when the mismatch is clearly larger than that, and sends the unclear cases to the TA.

4 Research question

If a checker cannot read charts perfectly, then for every claim it has to choose between deciding on its own and handing the claim to a person (selective prediction). Our project is basically about this choice.

Main question. Suppose the system flags a claim only when the gap between the claimed number and the measured number is larger than its own reading error, and abstains when it is not sure. Will it make fewer wrong flags than simpler approaches, given that all of them use the same model, a similar compute budget, and decide the same share of claims?

Each claim ends in one of four outcomes: accepted, flagged as a mismatch, "no matching figure", or passed to the TA as not sure. In the first three the system answers on its own ("no matching figure" is also an answer), and only the last one is handed over. We define

$$\text{wrong-flag rate} = \frac{\text{flagged claims that contained no error}}{\text{all flagged claims}}$$

$$\text{coverage} = \frac{\text{claims the system decided}}{\text{all claims}}$$

The wrong-flag rate is the false discovery rate of the flags. We also want to answer two smaller questions:

- How small an error can each approach catch?
- Does fine-tuning a small model for chart reading reduce the reading error enough that the system can decide more claims?

5 Why the problem matters

A comment like "this number does not match your figure" can cost a student marks, so it needs to be correct. If the tool is wrong, the student gets blamed for a correct sentence and TAs will stop trusting the tool. Checking by hand is tedious and people also miss things, so a tool that handles the clear cases and leaves only the doubtful ones for the TA would save a good amount of time.

Current models cannot be trusted with this on their own. On SciVer the best model reaches 77.7% accuracy whereas human experts reach 93.8% [1], and on MuSciClaims the best model reaches only 0.72 F1 [2].

6 What is missing in existing approaches

SciVer [1] and MuSciClaims [2] are benchmarks where claims are checked against the figures and tables of scientific papers. They show that models struggle, but they do not propose a method. ToolSciVer [3] gives the model tools for zooming and chart parsing and improves accuracy, but it still gives a verdict for every claim, and its authors mention uncertainty-aware verification as future work. A recent chart fact-checking system [4] votes over repeated answers and also answers every time. Berkane et al. [5] use the disagreement between repeated samples as a warning sign for chart reading, but note that this signal is not calibrated.

In the work we reviewed, we did not find a system which measures how accurate its own chart reading is and then uses that to choose between flagging a claim, accepting it, or handing it to a person. We also did not find one which reports how small an error it can catch.

7 Capabilities used and why the problem needs them

We use all five capabilities, but not equally. Most of the work is done by tool calling, reasoning and retrieval, while fine-tuning and the agent loop play a supporting role (Table 1). We will also run the system with each capability switched off, to see how much it contributes.

Capability	Why the problem needs it	How we measure it
Tool calling	The model is not reliable at reading exact values or doing arithmetic. The system zooms into the chart, converts it to a table, reads the axis labels with OCR and does the arithmetic in Python.	Share of tool calls that are valid and succeed, whether the right tool was chosen, and how often the system tries an action it is not allowed to take.
Reasoning	Each sentence has to be turned into something that can be checked: which line or bar, which point, what number, and how much rounding is acceptable.	Accuracy of this parsed form against the known answer. We also record at which step each error happens.
RAG (retrieval)	A sentence often does not say which figure it is about. The system has to find the right figure in the report, or notice that no figure covers the claim.	How often the correct figure is ranked first, and accuracy on the "no matching figure" cases.
Fine-tuning	Reading error limits everything else. We fine-tune a small model with LoRA so that it converts charts to tables more accurately.	Reading error before and after fine-tuning, and the change in coverage.
Agentic abilities	The system decides whether to stop, take another measurement, or hand the claim to the TA. It also remembers what it has already extracted from a report.	Tool calls per claim, how often it hands off, and whether the extra measurements change the outcome.

Table 1: The five capabilities in our project.

8 Proposed idea and architecture

The idea is to treat chart reading like any other measurement, i.e. it comes with an error margin. If the difference between the student's number and our reading of the chart is smaller than our reading error, the system should not call the sentence wrong. The steps are shown in Figure 1.

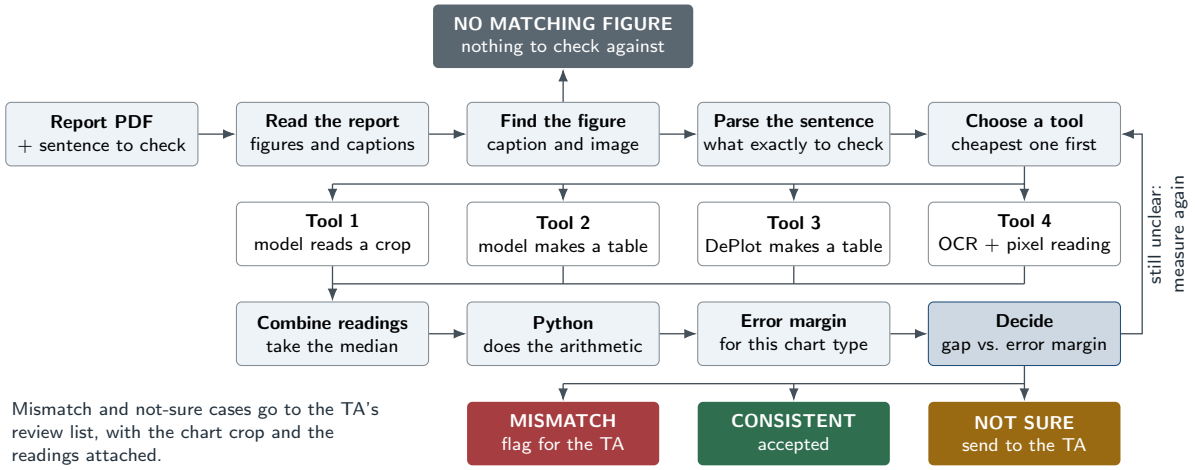


Figure 1: The proposed system. Each value is read with up to four different tools and the readings are combined. The decision compares the gap between the claim and the measurement with the error margin. Posting to the TA's review list is the only action the system can take.

1. *Find the figure.* The system looks for the figure the sentence refers to, using the figure number, the caption text and an image retrieval model. If no figure matches, the answer is "no matching figure".

2. *Turn the sentence into a check.* A model rewrites the sentence into a fixed form, i.e. which line or bar, which point, what number is claimed, and how much rounding the wording allows.

3. *Measure.* The values are read from the chart in up to four ways: a crop read by the vision-language model, a chart-to-table conversion by the same model, DePlot [6], and OCR with pixel measurement. The median is our estimate of the value. The disagreement between the tools does not change the estimate, it only decides how much the estimate can be trusted. All arithmetic is done in Python.

4. *Estimate the reading error.* On the calibration charts, where the true values are known, we compute the error of each reading as a fraction of the axis range, and group the errors by chart type and by how much the tools disagreed. In each group we take two percentiles: a higher one, U_{flag}, for flagging (starting with the 95th) and a lower one, U_{acc}, for accepting (starting with the 75th).

5. *Decide.* Let g be the gap between the claimed number and our estimate, and t the rounding allowed by the wording. The claim is flagged if $g > t + U_{\text{flag}}$ and accepted if $g \leq t + U_{\text{acc}}$. In between, the system takes another measurement with a different tool, up to 8 tool calls per claim (a starting value). If it is still unclear, the claim goes to the TA.

The rule is intentionally stricter for flagging, since a wrong flag is the more harmful mistake. The percentiles will be tuned so that the wrong-flag rate stays at or below 10%.

9 Expected novelty

We are not trying to build a better chart reader, since the model and the tools already exist. What we contribute is the way the decision is made and the way the system is evaluated.

First, the decision to flag, accept or abstain depends on a measured reading error, whereas the systems in Section 6 always give a verdict. Second, each value is read with tools of different kinds. The same model asked five times usually repeats the same mistake, but a pixel-based reader and a separate model make different mistakes, so their disagreement is more informative. Third, we report the smallest error that each system can catch, which we have not seen reported for chart claim checkers.

The individual pieces (repeated sampling, abstaining when unsure, etc.) are known techniques. What is new is combining them for this problem and evaluating the result mainly on wrong flags.

10 Dataset and evaluation environment

We build the main dataset ourselves, so that the correct answer for each sentence is known. We generate tables that look like student ML results (and reuse some from WB-ChartExtract [5]), plot them as bar and line charts in different styles, and put them into short PDF reports. We plan for around 240 reports.

The sentences to be checked are generated from the same tables. Some are true, including rounded ones. Some have a planted error of known size (from 2% to 50%) or a mix-up such as a wrong series. Some refer to something which no figure in the report shows.

The split is done by report and by underlying table, so nothing appears in two splits. Our planned sizes are 60 reports for estimating the reading error, 30 for development and 150 (about

1,000 sentences) for the final test. Fine-tuning uses a separate pool of about 6,000 charts. These sizes may be reduced after the pilot. If time permits, we will also try the main methods on a small set of real chart claims from MuSciClaims [2] and SciVer [1].

For the action side we use a small mock LMS in which the only allowed action is posting a flag for the TA. It also has two tools which the system must never use, and some reports contain hidden instructions, so we can count unsafe actions.

11 Baselines

All the baselines use the same vision-language model as our system.

1. *Direct verdict.* The model sees the figure and the sentence and gives its answer.
2. *Direct verdict with confidence.* Same as above, but the model abstains if its stated confidence is low.
3. *Majority vote.* The model is asked several times, and abstains if the answers disagree [7].
4. *Tools without our rule.* The model gets the same tools as our system, uses them freely, and gives its own verdict.

Baselines 2 to 4 can abstain, so we compare them with our method at the same coverage, with thresholds tuned on the same data.

12 Metrics

Our main numbers are:

- *Wrong-flag rate* and *coverage*, as defined in Section 4.
- *Coverage at a 10% wrong-flag rate*, which is the number we use to compare the methods.
- *Catch rate:* the share of planted errors that get flagged, for each error size, measured with the wrong-flag rate held at or below 10%. From this we read off the smallest error that a method catches at least half of the time.

We also report plain accuracy and F1 with abstention turned off. The measure for each capability is given in Table 1, and for safety we count how many times the system tries to use a forbidden tool.

13 Compute plan

The project is planned to run on the free GPUs from Kaggle or Colab (16 GB T4 class), and the main experiments do not depend on any API. We may use a hosted API model for a small side comparison on about 200 claims, but the provider is not decided yet. Table 2 gives our initial estimate of GPU time, which we will update after timing a small pilot.

Task	GPU-hours
Building the dataset, OCR and pixel measurements	0 (CPU only)
Retrieval index, DePlot runs, checking the chart reader	about 3
LoRA fine-tuning of the 4B model on about 6,000 charts, 2 epochs	4–6
Running all methods and ablations on the development and test sets	6–8
Repeated runs and spare capacity	6
Total	**about 19–23**

Table 2: Initial estimate of GPU time.

14 Expected risks and fallback plan

Risk	What we will do	Fallback
The task is too easy and the simple baselines already do well.	Run a pilot in week 2. If needed, make the charts harder: no value labels, more lines per chart, log axes.	Use the WB-ChartExtract charts, which are harder to read.
All tools misread a chart in the same way, so the error margin comes out too small.	Use tools of different kinds and check, for each chart type, how often the true value falls inside the margin.	Use wider margins where this happens and report it as a finding.
Error margins learned on our own charts do not carry over to charts from real papers.	Re-estimate the margins on a small set of about 50 real examples.	Report the gap between synthetic and real charts as a limitation.
The system abstains too often to be useful.	Let it take extra measurements before giving up, and report results at several wrong-flag targets.	Show the full trade-off between wrong flags and coverage.
Setup problems with the new model, or the work takes longer than planned.	Fix the software versions and test the whole pipeline in week 1.	Switch to an older model (Qwen2.5-VL-7B) and cut the repeated runs.

References

[1] C. Wang, Y. Shen, Z. Kuang, A. Cohan, Y. Zhao. SciVer: Evaluating Foundation Models for Multimodal Scientific Claim Verification. ACL 2025. arXiv:2506.15569.

[2] Y. K. Lal et al. MuSciClaims: Multimodal Scientific Claim Verification. IJCNLP-AACL 2025. arXiv:2506.04585.

[3] B. Zhou, P. Shi, R. Kamoi, N. Zhang, R. Zhang. ToolSciVer: Multimodal Scientific Claim Verification with Visual Tool Augmented Reinforcement Learning. arXiv:2607.16131, 2026.

[4] N. Verma, A. Mishra. Retrieval-augmented chart-grounded fact verification with self-consistent vision–language reasoning. *International Journal of Data Science and Analytics*, 2026. doi:10.1007/s41060-026-01277-5.

[5] T. Berkane, Q. Wang, M. S. Majumder. Self-Ensembling Vision-Language Models for Chart Data Extraction. arXiv:2605.27298, 2026.

[6] F. Liu et al. DePlot: One-shot Visual Language Reasoning by Plot-to-Table Translation. Findings of ACL 2023.

[7] X. Wang et al. Self-Consistency Improves Chain of Thought Reasoning in Language Models. ICLR 2023.